

Structure-Guided Retrieval with LLM-Directed Table-of-Contents Navigation and Precomputed Semantic Contagion: A Hybrid RAG Architecture

Mohamed MAJRI

September 15, 2026

Code and data: https://github.com/MM01984/structure_guided_retrieval_public

Abstract

Retrieval-augmented generation (RAG) systems commonly fail in one of two complementary ways: fine-grained embedding retrieval finds a relevant sentence but returns it stripped of the surrounding context needed to interpret it (the “needle in a haystack” problem), while coarse chunk-based retrieval loses the document’s global structure, producing answers that miss the broader picture a question actually calls for. We propose a hybrid RAG architecture that addresses both failure modes simultaneously through three components: (1) an LLM is given a flattened, enriched table of contents (TOC) of the corpus and decides itself which whole sections to retrieve verbatim, exploiting the LLM’s structural reasoning rather than embedding similarity; (2) the same LLM call independently proposes several *keyword combinations* – a principal phrasing plus optional synonym phrasings – each matched *independently* via its own BM25 query over paragraph-level units, with a per-combination quota governing how many passages each contributes, so that a single dominant concept cannot crowd out the others; (3) within a combination’s shortlisted blocks, candidate anchor units are ranked by a three-tier lexical-match strength (exact whole-word match of the principal phrasing, of a synonym phrasing, or a partial term match) before being expanded into a coherent, contiguous passage via *semantic contagion groups* – adjacency chains over sentence-embedding cosine similarity, precomputed entirely offline, so that no vector computation, and no comparison of any embedding against the question or its keywords, occurs at query time. We port the method to FinanceBench [Islam et al., 2023], a public benchmark of real SEC financial filings whose documents exhibit a genuine multi-level section hierarchy and whose gold answers are frequently synthesized, multi-step financial reasoning rather than short extractive spans – a corpus chosen specifically to match the method’s design intent, unlike flatter benchmarks with single-sentence answers. On the 150 publicly released FinanceBench questions, our method attains an Answer Accuracy exceeding, and a Page Recall far exceeding, the figures reported by HiREC [Choe et al., 2025], a peer-reviewed hierarchical-retrieval system built on fine-tuned cross-encoders, an iterative complementary-retrieval loop, and a two-model architecture (GPT-4o plus Qwen2.5-7B-Instruct) – while our own architecture uses a single, non-fine-tuned LLM in a single pass. From the composition and weighting of HiREC’s own published token-cost table, we further derive a lower bound, exceeding its own reported combined-benchmark average, on its per-query token consumption specifically on FinanceBench – a bound we can only establish indirectly, because token cost is reported inconsistently, or not at all, across the financial-QA literature we survey, and which already falls below our own directly measured figure, though whether HiREC’s true FinanceBench-specific cost does as well remains open.

1 Introduction

Retrieval-augmented generation systems are typically evaluated and designed around a single retrieval mechanism: split a corpus into fixed- or variable-size chunks, embed each chunk, and at query time retrieve the top- k chunks by cosine similarity to the embedded query. This design is simple and effective in many settings, but it is well known to suffer from two complementary failure modes.

First, the *needle-in-a-haystack* problem: a query term or concept may be phrased differently from the source text, or the single sentence that actually answers the question may be surrounded, in the retrieved chunk, by unrelated material – or worse, split across a chunk boundary that an arbitrary fixed-size split happened to introduce. Dense retrieval can locate the right neighborhood of the corpus but return a fragment that is technically similar yet practically unusable without its context.

Second, the *loss of global structure* problem: when a question calls for a synthesis across an entire section, chapter, or theme (“what does this text say about X, throughout”), chunk-level retrieval – built to find locally similar passages – has no mechanism to recognize that the correct answer requires assembling an entire structural unit of the document, and instead returns a handful of locally-similar but globally disconnected fragments.

We describe an architecture, developed independently, that treats these as two distinct problems requiring two distinct, complementary retrieval paths, orchestrated by a single upstream LLM planning call rather than an iterative agentic loop. Section 3 describes the architecture in full. Section 2 situates it against prior work. Section 4 reports an empirical evaluation on a public financial-document question-answering benchmark, chosen specifically because its documents and gold answers match the method’s design intent. Section 5 discusses limitations and threats to validity.

2 Related Work

LLM-directed structural navigation. The idea of an LLM navigating a tree of document summaries via iterative prompting, rather than embedding similarity, predates the RAG literature that popularized the term: LlamaIndex’s original Tree Index [Liu, 2022], released as “GPT Tree Index” in November 2022, already has the LLM descend a precomputed summary tree top-down, at each level answering a multiple-choice prompt over the child summaries and recursing into the selected branch until a leaf is reached. MemWalker [Chen et al., 2023] refines this idea with a formal academic evaluation and an explicit error-recovery mechanism absent from the original Tree Index: it treats the LLM as an interactive agent that can reason about its own navigation decisions and backtrack out of a branch that turns out, on reflection, not to contain the answer, rather than committing irrevocably to a single top-down descent. More recently, PageIndex [VectifyAI, 2025] builds a hierarchical “smart table of contents” over a document and lets an LLM reason over it directly, without any vector database, to navigate to the relevant section – a close cousin of the section-selection half of our architecture (Section 3.2). Where MemWalker’s navigation is iterative and stateful across several prompts (the LLM issues multiple prompts, descending – and potentially backtracking through – the tree step by step), and PageIndex’s public description does not detail a complementary lexical-anchoring mechanism for information absent from the TOC, our architecture performs the equivalent section-selection decision in a single LLM call, with no backtracking, and pairs it with an independent lexical retrieval path (Section 3.3) rather than relying on navigation alone to recover from a wrong turn.

Semantic chunking and neighbor expansion. A large body of RAG engineering practice groups sentences into chunks by splitting where adjacent-sentence embedding similarity drops below a threshold [Unstructured.io, 2025], and separately, expands a single embedding-matched sentence with a fixed-size window of neighboring sentences to preserve local coherence [Stepanov, 2025]. Our *contagion* mechanism (Section 3.4) is a variant of this second idea, but departs from it in two ways: the expansion window has *variable length*, determined by a chain of pairwise adjacent-similarity tests rather than a fixed neighbor count; and the adjacency chains are *precomputed once, offline, for the entire corpus* and merely looked up at query time, rather than computed on the fly around whichever sentence the query happens to match. A third, more fundamental departure, elaborated in Section 3.4, is that the embeddings never participate in *matching* at all – they are consumed exclusively to build this fixed adjacency structure ahead of time, never to score a candidate passage against the question.

Hybrid lexical-dense search. Combining a sparse retriever (BM25) with a dense retriever and fusing the results is standard practice [Prem AI, 2025], motivated by the observation that dense retrieval alone frequently misses queries anchored on exact lexical terms. Our architecture is a more categorical departure from this pattern than the term “hybrid” usually implies: it performs no dense retrieval *at query time* at all, in either of its two paths. The TOC path (Section 3.2) is a discrete, LLM-directed choice among section identifiers, not a similarity search; the lexical-anchor path (Section 3.3) is symbolic BM25 scoring [Robertson and Zaragoza, 2009] over keyword combinations proposed by the LLM. The only place an embedding appears anywhere in the online request path is as a static, precomputed lookup table – the contagion groups – consulted after a lexical match has already been found, purely to decide how far the returned passage should extend around that match, never to decide whether it is a match in the first place.

Financial document question-answering benchmarks. FinanceBench [Islam et al., 2023] evaluates open-book financial QA over real US SEC filings (10-K, 10-Q, 8-K, and earnings releases), with human-labeled Correct/Incorrect/Failed-to-answer judgments across eight retrieval configurations; its published results span a wide 9%–85% success range depending on configuration, and notably show that supplying an entire filing as long context can outperform naive chunked vector-store retrieval, a finding we return to in Section 4.2. Setty et al. [2024] report a smaller-scale follow-up study on the same public FinanceBench sample, comparing base retrieval, query expansion, and re-ranking configurations via an LLM-based evaluation score whose exact prompt and scale are not fully specified in that work. HiREC [Choe et al., 2025], accepted at ACL 2025 Findings, is the most methodologically precise of the three: it proposes a hierarchical document-then-passage retrieval pipeline with fine-tuned cross-encoders, an iterative evidence-curation loop that can issue complementary queries, and reports page-level Retrieval Recall/Precision against human-annotated evidence pages alongside Answer Accuracy, evaluated on a combined benchmark (LOFin) that includes FinanceBench as one of three constituent sources, with results broken out per source. We use HiREC as our primary quantitative point of comparison for this reason – precisely defined, reproducible metrics, open-source code, and a FinanceBench-specific results breakdown – while citing the other two for broader context on the benchmark’s history and known difficulty range.

3 Method

The architecture separates *offline preparation* (Section 3.1), performed once per corpus, from *online retrieval* (Sections 3.2–3.4), performed once per query via a single LLM call plus deterministic post-processing.

3.1 Offline preparation

Given a corpus of Markdown-structured documents (headers # through #####), two artifacts are built:

Table of contents. Every header line is recorded as a section with a stable identifier (its source line number), a cleaned title, a hierarchical breadcrumb path, and a line range extending to the next header of *equal or higher* level – so that a top-level section’s range recursively contains all its descendants. Concatenated per document, this yields a flattened text suitable for inclusion in an LLM prompt.

Retrieval index. Independently, the corpus is split into *blocks* (the text between one header and the *immediately next* header, of any level) and each block is further split into *units* (groups of one or more sentences, merged so that no unit falls below a minimum word count). Each unit is embedded with a sentence-embedding model. For each block, *contagion groups* are then precomputed: walking the block’s units in order, a cosine-similarity test is applied to each pair of *immediately adjacent* units; consecutive units whose similarity exceeds a threshold τ are chained into the same group, and a new group begins otherwise. This is a linear-time, adjacency-only computation, and its output – a partition of each block’s units into contiguous runs – is the only artifact from the embeddings retained downstream: no embedding vector, and no similarity computation, is needed once the groups are built. Finally, a BM25 index is built separately *per source document* (Section 4 explains why this matters for a corpus of independent, self-contained documents such as SEC filings, in contrast to a single corpus-wide index).

3.2 TOC-directed section retrieval

At query time, a single LLM call receives the user question together with the table of contents of the one document the question concerns, and returns, in a fixed text format: (i) a set of lexical keyword combinations (Section 3.3), and (ii) a ranked list of section identifiers the LLM judges directly relevant, drawn from the same TOC. The prompt explicitly instructs the LLM to prefer the *most deeply nested, most specific* matching section over a broad ancestor section whenever both would answer the question – a nested section is shorter and cheaper against the fixed token budget, whereas an ancestor section’s range recursively subsumes every one of its own descendants’ text (Section 3.1), so requesting it can silently consume the entire section budget on a single, overly broad extraction and starve every other requested section behind it. Because TOC sections are nested, a request for a top-level section still returns its entire subtree verbatim, unsegmented, up to the token budget, when the LLM does judge that the coarsest granularity is what the question calls for. This path is the one that supplies *global context*: it is exercised whenever a question asks about a document-level theme (a company’s overall liquidity profile, a filing’s stated risk factors) rather than a single isolated figure.

3.3 Lexical anchor retrieval

The same LLM call that selects TOC sections (Section 3.2) also proposes 1 to 6 *keyword combinations*, ordered by decreasing priority. Each combination consists of a *principal phrasing* – a

single term or a short multi-word expression whose words are meant to be searched *together* – and, optionally, one or more *synonym phrasings*: alternate wordings for the same concept, used when the question’s vocabulary is expected to diverge from the filing’s own (e.g. an informal ratio name versus the literal line-item label it decomposes into). This combination-based formulation, together with the per-combination scoring and quota described below, follows the design of a sibling deployment of the same base architecture on a different corpus (Solvency II regulatory text), which independently converged on the same approach in place of a single pooled bag of undifferentiated keywords.

Each combination is scored *independently* against the per-document BM25 index (Section 3.1): its principal and synonym phrasings are tokenized together into a single query, and the top-scoring blocks for that query alone – not for the pooled set of every combination’s terms – are shortlisted as candidates. Within a combination’s shortlisted blocks, every constituent unit is assigned a match tier: *tier 3* if every (non-trivial) word of the principal phrasing appears in the unit as a whole word; *tier 2* if it does not, but every word of some synonym phrasing does; *tier 1* if neither phrasing matches in full but at least one of their words appears as a whole word; *tier 0*, excluded, otherwise. Candidates are ranked by (tier, BM25 block score) and the top ones – up to a fixed *quota* for that combination’s priority rank – are accepted as anchors, deduplicated against blocks already covered by the TOC path (Section 3.2). Because the ranking and the quota are both scoped to a single combination, a concept that a naive pooled ranking would rank behind a more lexically frequent, unrelated one is guaranteed a bounded share of the retrieval budget regardless of how the other combinations score – this is the path that supplies the *needle*: a single number, ratio, or short clause the question asks for, wherever in the document it happens to sit, independent of which structural section it belongs to, and independent of how many other concepts the same question also requires.

3.4 Semantic contagion expansion

Each accepted anchor (Section 3.3) is a single unit that matched a keyword combination lexically – never an embedding-similarity match of any kind. The precomputed contagion group (Section 3.1) containing that unit is retrieved by a constant-time lookup, and its full span of units is included in the output, reconstructing a contiguous, coherent passage around the anchor rather than an isolated matched sentence. We emphasize the point because it is easy to elide in a hybrid-retrieval description and is, in our reading, the most distinctive property of this architecture relative to the broader hybrid lexical-dense literature (Section 2): **at no point in the online request path is any embedding compared against the question, against any keyword the LLM proposes, or against any other embedding computed at query time.** The sentence embeddings are consumed exactly once, entirely offline, to build a fixed partition of each block into adjacency-chained groups (Section 3.1); at query time they are not recomputed, not loaded into a vector index, and not involved in deciding *whether* a passage is relevant – that decision is made purely lexically, by BM25 and the tier rule of Section 3.3. The embeddings’ only role is to decide, after relevance has already been established some other way, *how far a passage should extend* around the point where relevance was found. This is a stronger claim than “the semantic component is precomputed”: it is that semantic similarity is never used as a relevance signal in this architecture at all, only as an offline-fixed expansion rule applied after a lexical or structural decision has already selected the anchor.

The two retrieval paths (Sections 3.2 and 3.3–3.4) are complementary by construction, not by tuning: one path is built to answer “what does this document say, broadly, about X” and the other to answer “what is the exact figure for Y”, and a single upstream LLM call decides, per question,

which of the two – or both – to invoke. A final LLM call combines both contexts to generate the answer (Section 4).

4 Experimental Setup

Corpus choice. The production system underlying this architecture serves a religious-text corpus with a deep, multi-level section hierarchy. An early port of the method to a benchmark with an almost flat document structure and single-sentence extractive gold answers produced results that, on inspection, failed to discriminate meaningfully between retrieval strategies of very different sophistication – a symptom, we concluded, of the benchmark rather than of any one method, and not a useful test of an architecture whose entire premise is that document structure and answer complexity both matter. We instead select FinanceBench [Islam et al., 2023]: real US SEC filings (10-K, 10-Q, 8-K filings, and earnings releases) whose native section structure (**PART I-IV**, **Item 1** through **Item 15**, plus company-specific subsections) is genuinely multi-level, and whose gold answers range from single-figure lookups to multi-step synthesized reasoning (e.g. computing and comparing several financial ratios across fiscal years) – matching, rather than being favorable to, the method’s original design intent.

Corpus preprocessing. We use the 150 questions and 84 underlying filings released as FinanceBench’s open-source sample. Filings are native-text PDFs (not scanned images); we convert them to Markdown, detecting section headers via a combination of regular-expression matching against the standard SEC **PART/Item** numbering and bold-font detection (verified, per filer, against the raw PDF’s font metadata, since some filers render these markers only as small running headers rather than in-body titles). Bold, all-capitals short lines are also the typographic convention SEC filers use for financial-statement line-item labels (e.g. **TOTAL REVENUES, FISCAL 2021, % CHANGE**) inside tables, which a naive bold-plus-short-line detector cannot distinguish from genuine subsection headings, fragmenting large tables into dozens of spurious one-line sections and diluting the table of contents with noise. We therefore suppress a candidate heading when it sits inside a locally dense run of other candidates, or immediately adjacent to a purely numeric/currency line – both reliable signatures of a table row rather than a structural heading – which we verified, by manual inspection, reduces spurious level-2 headings by roughly 5-to-1 on the most affected filings without a corresponding loss of genuine subsection titles. Because bold-font-detected headers alone still leave some filings with sparse structure – some filers do not visually repeat section titles in the body at all – large gaps between detected headers are further subdivided using an LLM asked to propose a short descriptive title per gap segment, so that every filing ends up with a genuinely multi-level, navigable hierarchy regardless of its native formatting quality. Each page boundary in the source PDF is preserved as an inline marker through this whole pipeline, stripped before indexing but retained for the page-level evidence metrics of Section 4.2.

Systems compared. *Proposed*, using the lexical-anchoring mechanism of Section 3.3 at a 4,000-token budget for TOC-section retrieval and a 3,000-token budget for BM25-anchor-plus-contagion retrieval (per-combination quotas [3, 2, 1, 1, 1, 1] by priority rank). We explored whether the per-combination quota was too tight and starving lower-priority concepts of retrieval budget by trying looser quotas ([4, 3, 2, 2, 1, 1]), first at the same 3,000-token BM25 budget and then at a raised 4,000-token budget; only the second variant was carried through to a full 150-question evaluation (the first was screened on a 15-question sub-sample only, where it appeared favorable – Section 5 explains why we do not report that sub-sample result as evidence). At full scale, the raised-

budget/looser-quota variant moved Page Recall and Page Precision by about a point each (in opposite directions) relative to the configuration reported here, while reducing Answer Accuracy by over 3 points (52.0% vs. 55.3%); we retain the tighter-quota, lower-budget configuration in Table 1 as the better Accuracy/Precision balance (Section 5 discusses this ablation further). A single non-fine-tuned LLM (`gemini-2.5-flash`, falling back to `gemini-3-flash-preview` on error) is used for every LLM call in the pipeline: TOC/keyword planning, the LLM-based gap-titling step of corpus preprocessing, and final answer generation.

Metrics. **Answer Accuracy:** numeric gold answers (a bare figure or percentage) are scored by tolerant numeric equality (2% relative tolerance, accounting for rounding); textual gold answers are scored by an LLM judge given the question, gold answer, and generated answer, returning a binary correctness verdict – in the spirit of, though not a character-for-character reproduction of, the FAMMA judging prompt HiREC itself uses [Choe et al., 2025]. **Page Recall / Precision:** FinanceBench’s public release does not itself carry page-level evidence annotations; we obtain them from HiREC’s own published LOFin dataset, which does, matched back to our FinanceBench question IDs. We report these under two distinct conventions, because our retrieval unit (a structural block or section, of variable length) does not coincide with a page the way HiREC’s own retrieval unit does (Section 4.2 explains why this distinction is not cosmetic): a *whole-page* convention, crediting any page touched by any amount of retrieved text as fully retrieved, and a *decimal* convention, crediting each page only in proportion to the fraction of its own text actually delivered to the LLM, computed from the precise character span of every retrieved unit against the page boundaries recorded during preprocessing. **Token consumption:** the true `total_token_count` reported by the Gemini API per call (which includes internal reasoning tokens not separately itemized by this API version), summed across every LLM call issued for a given question.

4.1 Results

System	Answer Accuracy	Page Recall	Page Precision	Tokens/query
Proposed	55.3%	62.0%	11.0%	10,858
HiREC [Choe et al., 2025]	50.0%	40.0%	– [‡]	>5,815 [†]

Table 1: Results on the 150-question public FinanceBench sample, at a 4,000-token TOC-section retrieval budget and a 3,000-token BM25-anchor-plus-contagion retrieval budget. Page Recall/-Precision use the decimal convention (Section 4); Section 4.2 also reports the whole-page convention. [†]Lower bound, not a FinanceBench-specific measurement – see Section 4.2 for the argument. [‡]HiREC reports Page Precision only on its combined three-dataset benchmark (21.79%), never broken out per source dataset, so no FinanceBench-specific figure is available to place in this cell.

Our method substantially outperforms HiREC’s published FinanceBench Answer Accuracy and Page Recall. Our own token cost (10,858/query) already exceeds the lower bound we derive for HiREC’s FinanceBench-specific cost (>5,815/query, Section 4.2); whether it exceeds HiREC’s true cost, which that lower bound does not pin down, remains open. Page Precision (11.0%) remains below HiREC’s own combined-benchmark figure (21.79%, not broken out per source dataset).

4.2 Comparison with HiREC, and a note on token-cost reporting

Why Page Precision differs so sharply by convention. Under the whole-page convention, our method’s Page Recall rises to 80.8%, because a structural block or TOC section that merely touches the edge of a page is credited with that entire page, regardless of how much of the page’s actual text was delivered (whole-page Answer Accuracy: 56.0%; whole-page Page Precision: 5.4%). HiREC’s own retrieval unit is the page itself (its passage retriever operates over PDF pages loaded directly via PyMuPDF, one page per candidate unit), so this ambiguity does not arise for it: a page is either retrieved as a whole or not at all, by construction. Reporting only the whole-page convention for our method would therefore not be a fair comparison – it inflates recall for any method whose retrieval granularity is finer than a page – which is why we report the decimal convention as our primary figure and the whole-page convention only as a secondary, explicitly-labeled data point.

Why we could not obtain a FinanceBench-specific token cost for HiREC, and why we report a lower bound instead. HiREC reports one token-cost table, averaged across its combined LOFin-1.4k benchmark (1,389 questions, 1,112 from FinQA, 150 from FinanceBench, the remainder from SEC-QA), not broken out per source dataset – despite HiREC’s own evaluation code distinguishing question source internally, indicating the breakdown was simply not published. The reported total of 5,815 tokens per query (4,604 retrieval, 1,211 generation, at input/output granularity: 4,291/313 and 1,052/159 respectively) is a weighted average over the three sources, and FinQA – 80% of the combined sample by count – supplies documents that are, by construction, pre-extracted single-page-plus-table excerpts rather than complete filings, categorically shorter than FinanceBench’s full, multi-hundred-page 10-K and 10-Q documents (a real filing in our own corpus averages 160 pages). SEC-QA, the remaining minority source, does use complete long filings comparable in scale to FinanceBench. A weighted average dominated (80% by count) by a systematically cheaper component can only sit below the true cost of its more expensive components: if FinQA’s per-query cost is categorically lower than that of the two long-filing sources, then FinanceBench’s own average – pooled with SEC-QA’s, both drawn from the more expensive tail of the distribution the reported 5,815 was averaged over – must itself exceed that overall average, not merely approach it. We therefore report **>5,815 tokens/query** in Table 1 as a lower bound for HiREC on FinanceBench specifically, derived directly from the arithmetic of the published average and its known composition, rather than as a rough approximation; we do not have the data to state a specific upper value.

A broader observation. None of the three financial-QA papers we survey (Islam et al., 2023; Setty et al., 2024; Choe et al., 2025) reports token or dollar cost at the same per-benchmark granularity as its accuracy figures; where a cost table exists at all (only in HiREC), it is aggregated across benchmarks rather than broken out per one. We consider this a genuine gap in current reporting practice in this literature: an accuracy improvement bought with several times the token budget is a different result from the same improvement at equal or lower cost, and a reader cannot currently distinguish the two from the numbers these papers publish. We report per-query token consumption as a first-class, per-benchmark metric in Table 1 for this reason, not as an afterthought.

5 Discussion and Limitations

Quota tuning: a precision/recall trade-off, not a free improvement. The per-combination quota mechanism of Section 3.3 is a genuine improvement in mechanism precision over a pooled,

boolean-partitioned ranking (a passage’s relevance to a specific concept is verified compositionally, not by whichever single term happens to score highest in a pooled ranking), and it measurably improves Page Precision and token efficiency relative to that alternative. It does not, however, uniformly improve retrieval effectiveness on this specific benchmark for every quota setting: a case inspection on a question requiring two distinct concepts (a revenue figure and a balance-sheet asset figure, to compute a turnover ratio) showed the correct balance-sheet block present among the BM25 candidates for its concept, at a valid lexical-match tier, but excluded by that concept’s quota, which two other, less relevant, higher-ranked candidates had already filled. Motivated by this diagnosis, we tried loosening the per-combination quota (from $[3, 2, 1, 1, 1, 1]$ to $[4, 3, 2, 2, 1, 1]$), first screened on a 15-question sub-sample where it appeared to improve Answer Accuracy and Page Recall together, then combined with a raised BM25 budget (3,000→4,000 tokens) and validated at full scale: at $n = 150$, this variant reduced Answer Accuracy (52.0% vs. 55.3%) while leaving Page Recall and Page Precision within a point of the configuration we report in Table 1. We did not carry the looser-quota, unchanged-budget variant through a full 150-question run, precisely because its favorable 15-question result was the kind of small-sample signal the next paragraph argues is unreliable at this benchmark size, and the one variant we did validate at full scale moved in the opposite direction on Accuracy. We read this as evidence that the per-concept quota is a real precision/recall trade-off on this corpus, not a parameter with a single strictly-better setting, and we report the configuration in Table 1 as the best fully-validated Accuracy/Precision balance we found, not as a dominant setting on every metric.

Filer-level preprocessing heterogeneity. SEC filings from different companies (and, presumably, different preparing law firms) vary substantially in whether section titles are rendered as visible in-body headings or only as small running headers – a real, filer-dependent property of the corpus, not an artifact of our specific header-detection heuristic, confirmed by inspecting the underlying PDF font metadata directly. Our LLM-based gap-filling step (Section 4) mitigates this but does not eliminate variance in structural quality across the 84 filings in the sample. The table-row/heading disambiguation of Section 4 is a heuristic density/adjacency rule, not a structural table parser; it is tuned by manual inspection on the most visibly affected filings and is not independently validated filer-by-filer.

Sample size and non-determinism. Repeated runs of the same configuration on the same question sample produced Answer Accuracy figures varying by several points between runs, attributable to LLM generation and judging temperature rather than to any change in the method or corpus – expected at $n = 150$, and considerably more pronounced at the $n = 15$ sub-sample we used during early iteration of the quota ablation above. The quota-loosened, raised-budget variant’s 15-question pilot suggested a Page Recall (and Answer Accuracy) improvement that not only failed to hold but reversed once measured at full scale ($n = 150$). We treat single-point comparisons, including our own headline figures, as indicative rather than exact, and specifically caution against tuning retrieval hyperparameters against sub-samples much smaller than the full evaluation set on a benchmark of this size.

Metric fidelity to HiREC. Our textual-answer LLM judge is written in the spirit of, but is not a verified reproduction of, HiREC’s FAMMA-based judging prompt; our numeric-tolerance rule (2% relative) is our own choice, not taken from either compared paper. Both choices affect Answer Accuracy in ways we have not attempted to isolate from the underlying retrieval quality.

6 Conclusion

We have described a hybrid RAG architecture combining LLM-directed navigation of a corpus’s table of contents with a lexical-anchoring mechanism – per-concept BM25 queries, tiered whole-word matching, and per-concept retrieval quotas – and precomputed semantic-contagion expansion, and evaluated it on FinanceBench, a benchmark whose genuinely multi-level document structure and frequently synthesized, multi-step gold answers match the architecture’s design intent. Against HiREC, a peer-reviewed, more heavily engineered comparison point built on fine-tuned cross-encoders, a two-model architecture, and an iterative complementary-retrieval loop, our considerably simpler architecture – a single non-fine-tuned LLM, a single retrieval pass, no reranker training – exceeds Answer Accuracy and Page Recall at a directly measured token cost that already exceeds HiREC’s own derived FinanceBench-specific lower bound, though whether it exceeds HiREC’s true cost remains open (Section 4.2); our own Page Precision remains below HiREC’s combined-benchmark figure. We read this, together with the mixed result of our own quota-tuning ablation (Section 5), as evidence that the architecture’s core design decision – one retrieval path for a question’s global context, exercised through LLM-directed structural navigation, and a second, independent path for the specific figure the question actually asks for, exercised through per-concept lexical anchoring and offline-precomputed semantic expansion that never compares an embedding against the query at request time – is doing useful work on a genuinely difficult benchmark, without requiring the additional machinery a more heavily engineered system brings to the same task, and that its remaining hyperparameters (per-concept quotas, retrieval budgets) trade Accuracy against Precision rather than offering a free improvement in both.

References

- Howard Chen, Ramakanth Pasunuru, Jason Weston, and Asli Celikyilmaz. Walking down the memory maze: Beyond context limit through interactive reading. *arXiv preprint arXiv:2310.05029*, 2023.
- Jaeyoung Choe, Jihoon Kim, and Woohwan Jung. Hierarchical retrieval with evidence curation for open-domain financial question answering on standardized documents. In *Findings of the Association for Computational Linguistics: ACL 2025*, 2025.
- Pranab Islam, Anand Kannappan, Douwe Kiela, Rebecca Qian, Nino Scherrer, and Bertie Vidgen. Financebench: A new benchmark for financial question answering. In *arXiv preprint arXiv:2311.11944*, 2023.
- Jerry Liu. Gpt tree index (llamaindex): Tree index. <https://gpt-index.readthedocs.io/en/v0.6.16/reference/indices/tree.html>, 2022. First released to GitHub November 2022 under the name GPT Tree Index. Accessed 2026-09-03.
- Prem AI. Hybrid search for rag: Bm25, splade, and vector search combined. <https://www.prem.ai/blog/hybrid-search-for-rag-bm25-splade-and-vector-search-combined/>, 2025. Accessed 2026-08-29.
- Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- Spurthi Setty, Harsh Thakkar, Alyssa Lee, Eden Chung, and Natan Vidra. Improving retrieval for rag based question answering models on financial documents. 2024.

Sander Stepanov. Precision meets context: Sentence chunking and neighbor expansion in rag. <https://medium.com/@sander.stepanov/precision-meets-context-sentence-chunking-and-neighbor-expansion-in-rag-25cf5dd57f1e>, 2025. Accessed 2026-08-29.

Unstructured.io. Semantic chunking for rag: How it works and when to use it. <https://unstructured.io/insights/semantic-chunking-for-rag>, 2025. Accessed 2026-08-29.

VectifyAI. Pageindex: Next-generation vectorless, reasoning-based rag. <https://pageindex.ai/blog/pageindex-intro>, 2025. Accessed 2026-08-29.